

Week 2 Day 1 — Agent Foundations: Reasoning Loops, Tool Calling & Raw Python Agents

Overview

An agentic system differs from a traditional chatbot or workflow because it can make decisions about what actions to take, use external tools, observe their results, and continue working toward a goal. A chatbot primarily responds to user messages, while a workflow follows a predefined sequence of steps. An agent, on the other hand, can dynamically decide which tool or action is required based on the current situation. Agents are useful for multi-step and dynamic tasks, but they can be unnecessary for simple, deterministic problems because they introduce additional latency, API costs, complexity, and possible failure points.

ReAct Loop

The agent in this task follows the basic ReAct pattern: **Reason → Act → Observe → Repeat**. The model first determines what needs to be done, then selects an appropriate tool and provides its arguments. The Python program executes the selected tool and returns the result as an observation. The model then uses that observation to decide whether another action is necessary or whether it can provide the final answer.

The loop can be represented as:

User Request → Model Reasoning → Tool Selection → Tool Execution → Observation → Model Reasoning → ... → Final Answer

This approach allows the agent to solve tasks that require multiple steps. For example, the implemented agent was tested with a request to compare the weather in London and Tokyo. It first called the weather tool for both cities, observed that Tokyo was 26°C and London was 18°C, then called the calculator tool to calculate the difference. Finally, it produced the answer that Tokyo was warmer by 8°C.

Tool Schemas and Tool Calling

Two tools were implemented: a **weather lookup tool** and a **calculator tool**. Each tool has a defined name, description, and input schema so that the model understands what the tool does and what arguments it requires. The weather tool accepts a city and temperature unit, while the calculator accepts an arithmetic expression.

Tool descriptions are important because they guide the model when deciding which tool should be used. Clear schemas also reduce incorrect or incomplete tool arguments and make tool execution predictable. The implementation uses the **Gemini API** for model interaction and manually executes the selected functions in Python before returning their results to the model.

Raw Python Agent and Memory

The agent was implemented without an agent framework using a Python while loop. Each iteration sends the current conversation to the model, checks whether the model requested a tool, executes the tool, adds the tool result to the conversation, and then continues the loop. A `max_iterations` limit prevents the agent from running indefinitely.

The implementation also maintains simple working memory containing the current iteration, tool calls, and observations. This is different from conversation memory: conversation memory stores the broader message history, while working memory stores temporary information needed to complete the current task. Logging was added to show the agent's steps, selected tools, arguments, observations, and final response.

Failure Modes and Mitigations

The agent was deliberately tested with invalid tool usage. Attempting to call an undefined `currency_lookup` tool produced an Unknown tool error, while providing incomplete arguments to the weather tool produced a missing city argument error. These experiments demonstrate why agent systems need safeguards.

Important failure modes include infinite loops, hallucinated or unknown tools, incorrect tool arguments, silent tool errors, unsafe tool inputs, and API/provider failures. Mitigations include setting a maximum iteration limit, restricting execution to registered tools, validating arguments against schemas, catching and returning structured errors, restricting dangerous operations, and using bounded retries with backoff for provider failures.

Conclusion

This task demonstrated how an agent can be built from scratch using a simple reasoning-and-action loop rather than relying on frameworks such as LangChain, LangGraph, or CrewAI. Building the agent manually makes the underlying process clear: the model decides what to do, Python executes the selected tool, the result becomes an observation, and the cycle continues until a final answer is possible. Frameworks become valuable as systems grow because they provide reusable components for state management, tool registration, retries, branching, persistence, tracing, and human approval. The completed implementation demonstrates the core concepts required to understand and build more advanced agentic systems.